Distributed Systems Project Ideas (Spring Boot + Kafka + Redis + Microservices)

Goal

Build real-world backend systems that solve practical problems while demonstrating production-grade distributed system concepts such as:

- Microservices
- Event-Driven Architecture
- Kafka
- Redis
- API Gateway
- Rate Limiting
- Retry & Exponential Backoff
- Circuit Breaker
- Distributed Locking
- Saga Pattern
- Outbox Pattern
- Dead Letter Queue (DLQ)
- Idempotency
- Distributed Tracing
- Monitoring
- Containerization
- Kubernetes

Project 1: Digital Payment Gateway (Recommended)

Problem Statement

Businesses need a secure and scalable platform to accept payments from multiple banks and payment providers while handling failures, retries, asynchronous callbacks, refunds, and reconciliation.

Examples:

- Stripe
- Razorpay
- Cashfree
- PhonePe Payment Gateway

Functional Requirements

Merchant

- Merchant registration
- API key generation
- Webhook configuration

Payment

- Create payment
- Payment authorization
- Capture payment
- Refund payment
- Transaction history

Bank Integration

- Multiple bank connectors
- Retry failed requests
- Handle asynchronous callbacks

Notification

- Email
- SMS
- Webhooks

Dashboard

- Merchant dashboard
- Transaction reports
- Settlement reports

Non-Functional Requirements

- High Availability
- Fault Tolerance
- Scalability
- Security
- Audit Logging
- Monitoring

Technologies

- Spring Boot
- Spring Cloud Gateway
- Kafka
- Redis

- PostgreSQL
- Docker
- Kubernetes
- Prometheus
- Grafana
- OpenTelemetry
- ELK Stack

Microservices

- API Gateway
- Auth Service
- Merchant Service
- Payment Service
- Bank Integration Service
- Notification Service
- Ledger Service
- Audit Service
- Analytics Service

Architecture

Client

↓

API Gateway

↓

Authentication

↓

Payment Service

↓

Kafka

↓

Ledger Service

↓

Notification Service

↓

Audit Service

↓

Analytics Service

↓

Bank Integration Service

↓

Bank APIs

Distributed System Concepts

- Kafka
- Redis
- Saga Pattern
- Outbox Pattern
- Circuit Breaker
- Retry
- DLQ
- Rate Limiting
- Distributed Lock
- Idempotency
- Eventual Consistency

Scalability

Payment Service scales independently.

Notification Service can consume millions of Kafka events without affecting payments.

Project 2: Food Delivery Platform

Problem Statement

Restaurants need an online ordering platform capable of handling thousands of concurrent users while ensuring inventory consistency and reliable order processing.

Examples

- Swiggy
- Zomato

Features

- Restaurant Management
- Menu
- Cart
- Orders
- Payment
- Delivery Partner
- Live Tracking
- Ratings

Microservices

- User Service
- Restaurant Service
- Cart Service
- Order Service
- Payment Service
- Delivery Service
- Notification Service

Technologies

Spring Boot

Kafka

Redis

PostgreSQL

WebSocket

ElasticSearch

Architecture

Customer

↓

Gateway

↓

Order Service

↓

Kafka

↓

Inventory Service

↓

Payment

↓

Delivery Assignment

↓

Notification

Distributed Concepts

- Saga Pattern
- Kafka
- Redis
- Distributed Lock
- Retry
- Circuit Breaker
- CQRS
- Event Sourcing

Project 3: Ride Booking Platform

Problem Statement

Provide fast ride booking with driver matching, location tracking, pricing, and payments.

Examples

Uber

Ola

Lyft

Features

- Driver Registration
- Passenger Registration
- Ride Request
- Driver Matching
- Live Tracking
- Payments
- Ratings

Technologies

Spring Boot

Kafka

Redis GeoSpatial

WebSocket

MongoDB

PostgreSQL

Services

- User Service
- Driver Service
- Ride Service
- Pricing Service
- Notification Service
- Payment Service

Distributed Concepts

- Geo Search

- Kafka
- WebSockets
- Redis
- Distributed Cache
- Retry
- Rate Limiter

Project 4: E-Commerce Platform

Problem Statement

Build a scalable marketplace capable of handling product catalogs, inventory, payments, and shipping.

Examples

Amazon

Flipkart

Features

- Product Catalog
- Search
- Cart
- Orders
- Payments
- Inventory
- Shipping

Microservices

- Product Service
- Inventory Service
- Cart Service
- Order Service
- Payment Service
- Shipping Service
- Notification Service

Distributed Concepts

- CQRS
- Event Sourcing
- Kafka

- Redis
- Distributed Lock
- Outbox Pattern
- Saga Pattern

Project 5: Stock Trading Platform

Problem Statement

Provide real-time stock trading while maintaining consistency and handling thousands of simultaneous orders.

Examples

Zerodha

Groww

Upstox

Features

- Portfolio
- Buy
- Sell
- Watchlist
- Real-time Prices
- Order Book

Technologies

Kafka

Redis

WebSocket

TimescaleDB

PostgreSQL

Concepts

- Event Streaming
- Kafka
- Matching Engine
- CQRS
- Event Sourcing
- Distributed Cache

Project 6: Multi-Bank Payment Switch

Problem Statement

Route financial transactions to multiple banks while providing retries, failover, reconciliation, and status tracking.

Examples

NPCI

Visa

Mastercard

Features

- Routing Engine
- Retry
- Timeout
- Failover
- Settlement
- Reconciliation

Services

- Routing Service
- Payment Service
- Bank Connector Service
- Audit Service
- Ledger Service

Distributed Concepts

- Kafka
- Retry
- Circuit Breaker
- Idempotency
- Saga Pattern
- Distributed Lock

Project 7: Distributed Job Scheduler

Problem Statement

Execute millions of scheduled jobs reliably across multiple worker nodes.

Examples

Temporal

Quartz

Airflow

Features

- Job Scheduling
- Retry
- Failure Recovery
- Worker Registration
- Cron Support

Technologies

Kafka

Redis

PostgreSQL

Concepts

- Leader Election
- Distributed Lock

- Retry
- DLQ
- Event Driven Workers

Project 8: Notification Platform

Problem Statement

Provide one unified platform capable of sending notifications across multiple communication channels.

Examples

Firebase

Twilio

AWS SNS

Channels

- Email
- SMS
- Push Notification
- WhatsApp

Services

- Notification Service
- Template Service
- Email Worker
- SMS Worker
- Push Worker

Concepts

- Kafka
- Retry
- DLQ
- Rate Limiter
- Redis Cache

Project 9: Cloud File Processing Platform

Problem Statement

Automatically process uploaded files by scanning, transforming, indexing, and storing them.

Examples

Dropbox

Google Drive

Features

- Upload
- Virus Scan
- OCR
- Compression
- Thumbnail Generation
- Metadata Extraction

Pipeline

Upload

↓

Kafka

↓

Virus Scanner

↓

OCR

↓

Thumbnail

↓

Storage

↓

Notification

Concepts

- Kafka
- Async Processing
- Retry
- DLQ
- Event Pipeline

Project 10: Event Ticket Booking Platform

Problem Statement

Sell event tickets without allowing double booking even under heavy traffic.

Examples

BookMyShow

Ticketmaster

Features

- Browse Events
- Seat Selection
- Reservation
- Payment
- Cancellation

Concepts

- Redis Distributed Lock
- Kafka
- Queue
- Retry
- Inventory Reservation

Common Technology Stack

Backend

- Java 21
- Spring Boot
- Spring Security
- Spring Cloud
- Spring Data JPA

Database

- PostgreSQL
- MySQL

Messaging

- Apache Kafka

Cache

- Redis

API Gateway

- Spring Cloud Gateway

Authentication

- JWT
- OAuth2

Monitoring

- Prometheus
- Grafana

Logging

- ELK Stack

Distributed Tracing

- OpenTelemetry
- Jaeger

Deployment

- Docker
- Kubernetes
- Helm

CI/CD

- GitHub Actions
- Jenkins

Distributed System Concepts to Implement

Every project should implement as many of these concepts as applicable:

- API Gateway
- Service Discovery
- Config Server
- Event-Driven Architecture
- Kafka Topics & Consumer Groups
- Redis Caching
- Redis Distributed Locking
- Idempotency
- Retry with Exponential Backoff
- Circuit Breaker
- Bulkhead Pattern
- Rate Limiting
- Saga Pattern
- Outbox Pattern
- Dead Letter Queue (DLQ)
- Eventual Consistency
- CQRS
- Event Sourcing (where appropriate)
- Distributed Tracing
- Health Checks

- Metrics Collection
- Audit Logging
- Graceful Shutdown
- Horizontal Scaling
- Auto Scaling
- Blue-Green Deployment
- Rolling Deployment

Recommended Learning Order

1. Notification Platform
2. Distributed Job Scheduler
3. Digital Payment Gateway
4. Multi-Bank Payment Switch
5. Food Delivery Platform
6. E-Commerce Platform
7. Ride Booking Platform
8. Event Ticket Booking Platform
9. Cloud File Processing Platform
10. Stock Trading Platform

This progression starts with simpler event-driven systems and gradually introduces more advanced distributed-system challenges such as orchestration, consistency, routing, and real-time processing.